# Codigo Py Quest BC2

```
import tkinter as tk
From tkinter import messagebox, simpledialog
import os
import random

Rosa_FONDO = "#ffe4f0"
ROSA_FUERTE = "#ff5fa2"
RosA_BOTON = "#ff8fc1"
LETRA = "Comic Sans MS"
#Clase que guarda y revisa los usuarios en un archivo de texto
Class Usuarios:
   #CREA la lista de usuarios y la llena con lo que hay en el archivo
   def __init__(self):
       self.archivo = "Usuarios.txt"
       self.lista = {}
       self.cargar()
   #Lee los usuarios que ya están guardados en el archivo
   def cargar(self):
   if os.path.exists(self.archivo):
      archivo = open(self.archivo, "r")
      For linea in archivo:
          datos = linea.strip().split(",")
          if len(datos) == 3:
            self.lista[datos[0]] = [datos[1], datos[2]]
      archivo.close()
   #Guarda un usuario nuevo en el archivo
   def guardar(self, usuario, clave, palabra):
       self.lista[usuario] = [clave, palabra]
       archivo = open(self.archivo, "a")
       archivo.write(usuario + "," + clave + "," + palabra + "\n")
       archivo.close()
   #Revisa si el usuario existe y si la contraseña es correcta
   def revisar(self, usuario, clave):
       if usuario in self.lista and self.lista[usuario][0] == clave:
          return True
       return False
#Clase de la ventana del login
Class Login:
#Arma la ventana rosita con las cajas de texto y los botones
def__init__(self, ventana):
   self.ventana = ventana
   self.usuarios = Usuarios()
   self.intentos = 0
   self.burlas = 0
   self.max_burlas = 5
   self.dezcansando = False
   self.color_actual = 0
   self.colores = ["#ff5fa2", "#ff8fc1", "#c77dff", "#ffb3d1"]
   self.frases = ["¡No me atrapas!", "Jeje ♥", "¿Me buscabas?", "Muy lenta", "Bruquito", "Casi"]

   Ventana.title("Login")
   Ventana.geometry("360x570")
   Ventana.configure(bg=RosA_FONDO)
   Ventana.resizable(False, False)
```

```
Sel. decoracion = tk. Label (Ventana, texT= "❀ ♥ ❀ ♥ ❀ ♥ ❀" bg = RO
SA_ FONDO, Fg = ROSA_ FUERTE, Font = (LETRA, 14))
    Self. decoracion. Pack (pady = (10,0))

    tk. label (Ventana, text = "✿ LOGIN ✿", bg = ROSA_ FONDO, Fg = RO SA _ FUERTE, Font = (-
    -LETRA, 20, "bold")). pack()

    Sel.mensaje = tk. Label (Ventana, text = " ", bg = ROSA_FONDO, Fg = "#d6 336C", Font = (LETRA, 10))
    Self. mensaje. pack (Pady = 5)

    tk. Label (Ventana, text = "Usuario ★", bg = ROSA_FONDO, Fg = ROSA_ FUERTE, Font = (LETRA, 11)). PACK()
    Self. Caja_ Usuario = tk. Entry (Ventana, Font = (LETRA, 11), Justify = "center", relief = "flat",
                                                highlightthickness = 2, highlightback
ground = ROSA_BOTON, highlightcolor = ROSA_FUERTE)
        Self. Caja_ Usuario. pack (Pady = 3)
        tk. Label (Ventana, text = "Contraseña ★", bg = ROSA_ FONDO, Fg = ROSA_ FUERTE, Font = (LETRA, 11). -
        - pack()
Self. Caja_ Clave = tk. Entry (Ventana, show = "•", Font = (LETRA, 11), Justify = "center", relief = "flat"
                                                highlightthickness = 2, highlightbackgr
ound = ROSA_BOTON, highlightColor = ROSA_ FUERTE)
        Self. Caja_ Clave. Pack (Pady = 3)
        Sel. recordar = tk. Intvar ()
        tk. checkbutton (Ventana, text = "Recuerdame", variable = Self. recordar, bg = ROSA_ FONDO, Fg = ROSA_ FUERTE,
                                                selectcolor = "White", activebackground = ROSA_ FONDO,
        activeForeground = ROSA_ FUERTE,
                                Font = (LETRA, 10)). Pack (Pady = 3)

        Self. Contenedor = tk. Frame (Ventana, width = 340, height = 160, bg = ROSA_ FONDO)
        Self. Contenedor. Pack()

        Self. boton_ entrar = tk. Button (Self. Contenedor, text = "Entrar ♥", width = 14, command = self. -
        -entrar,
                                bg = ROSA_ BOTON, Fg = "White", activebackground = ROSA_ FUERTE,
                                activeforeground = "White", relief = "flat", Font = (LETRA, 10, "bold"))

        Self. boton_ entrar. place (relx = 0,5, rely = 0,5, anchor = "center")
        Self. boton_ entrar. bind ("< Enter >", self. Escapar)
        Self. Contenedor. bind ("< Motion >", self. mouse_ Cerca)

        Self. caja_ usuario. bind ("< keyRelease >", Self. revisar_ Cajas)
        Self. caja_ clave. bind ("< keyRelease >", Self. revisar_ Cajas)

        Self. Crear_ boton_ bailarin ("¿Olvidaste tu contraseña ❀", self. olvide_ clave)
        Self. crear_ boton_ bailarin ("¿No tienes cuenta? Registrate ♥", Self. registrar)

        Self. decoracion_ abajo = tk. Label (Ventana, text = "♥ ❀ ★ ❀ ♥", bg = ROSA_ FONDO, Fg = ROSA_ -
        FUERTE, Font = (LETRA, 14))
        Self. decoracion_ abajo. Pack (Pady = 5)

        Self. Cargar_ recordado ()
        Self. Poner_ foco ()
#Crea un botón que tiemble cuando el mouse pase por encima
```

Scanned with CamScanner

```
def crear_boton_bailarin(self, texto, comando):
    marco = tk.Frame(self.ventana, width=320, height=40, bg=ROSA_FONDO)
    marco.pack()

    boton = tk.Button(marco, text=texto, command=comando, bg="white", fg=ROSA_FUERTE,
                      activebackground=ROSA_BOTON, activeforeground=
    ="white", relief="flat", font=(LETRA, 10))
    boton.place(relx=0.5, rely=0.5, anchor="center")
    boton.bind("<Enter>", lambda evento: self.temblar(boton, 0))

# Mueve el botón de lado a lado varias veces, como si temblara
def temblar(self, boton, paso):
    movimientos = [-8, 8, -6, 6, -4, 4, -2, 2, 0]
    if paso < len(movimientos):
        boton.place(relx=0.5, rely=0.5, x=movimientos[paso], anchor="center")
        self.ventana.after(40, self.temblar, boton, paso+1)

# Cambia el color de los flores cada medio segundo
def parpadear(self):
    self.color_actual = (self.color_actual + 1) % len(self.colores)
    self.decoracion.config(fg=self.colores[self.color_actual])
    self.decoracion_abajo.config(fg=self.colores[self.color_actual])
    self.ventana.after(500, self.parpadear)

# Si el mouse se acerca al botón "Entrar", el botón huye antes de que lo toque
def mouse_cerca(self, evento):
    centro_x = self.boton_entrar.winfo_x() + self.boton_entrar.winfo_width() /2
    centro_y = self.boton_entrar.winfo_y() + self.boton_entrar.winfo_width()/2 +30:
    if abs(evento.y - centro_y) < self.boton_entrar.winfo_height()/2 +30:
        self.escapar(evento)

# el botón "Entrar" huye a un lugar al azar, lejos del mouse
def escapar(self, evento):
    if self.desanimando:
        return
    vacios = self.caja_usuario.get().strip() == "" or self.caja_clave.get()==""
    if vacios or self.burlas < self.max_burlas:
        if vacios:
            self.mensaje.config(text="Llena los campos antes de continuar :)")
        else:
            self.burlas = self.burlas + 1
            self.mensaje.config(text="Eres tortuga ya (" + str(self.burlas) + "/" +
            str(self.max_burlas) + ")")
    ancho = self.boton_entrar.winfo_width()
    alto = self.boton_entrar.winfo_height()
    mouse_x = self.contenedor.winfo_pointerx() - self.contenedor.winfo_rootx()
    mouse_y = self.contenedor.winfo_pointery() - self.contenedor.winfo_ro

    for intento in range(30):
        x = random.randint(ancho//2, 340 - ancho//2)
        y = random.randint(alto//2, 160 - alto//2)
```

```
            if abs(x-mouse_x) > ancho or abs(y_mouse_y) > alto + 30:
                break

            self.boton_entrar.place(relx=0, rely=0, x=x, y=y, anchor="center")
            self.boton_entrar.config(text=random.choice(self.frases))
            self.ventana.after(200, self.terminar_descanso)
        else:
            self.boton_entrar.config(text="ya, entra v")
            self.mensaje.config(text="")

    # Deja que el botón vuelva a escaparse después de un momento
    def terminar_descanso(self):
        self.descansando = False

    # Si se borra alguna caja, el botón vuelve a burlarse desde 0

    def revisar_cajas(self, evento):
        if self.caja_usuario.get().strip() == "" or self.caja_clave.get() == "":
            self.burlas = 0

    # Si alguien marcó "Recordarme", escribe su usuario en la caja
    def cargar_recordado(self):
        if os.path.exists("recordado.txt"):
            archivo = open("recordado.txt", "r")
            usuario = archivo.read().strip()
            archivo.close()
            if usuario != "":
                self.caja_usuario.insert(0, usuario)
                self.recordar.set(1)

    # Guarda el usuario si marcó "Recordarme", o lo borra si no
    def guardar_recordado(self, usuario):
        archivo = open("recordado.txt", "w")
        if self.recordar.get() == 1:
            archivo.write(usuario)
        archivo.close()

    # Revisa los datos cuando se oprime "Entrar"
    def entrar(self):
        usuario = self.caja_usuario.get().strip()
        clave = self.caja_clave.get()

        if usuario == "" or clave == "":
            self.mensaje.config(text="Llena todos los campos")
            return

        if self.usuarios.revisar(usuario, clave):
            self.intentos = 0
            self.burlas = 0
            self.boton_entrar.config(text="Entrar")
            self.mensaje.config(text="")
            self.guardar_recordado(usuario)
            messagebox.showinfo("Bienvenido", "Hola " + usuario + ", entraste")
```

Scanned with CamScanner

```
else:
Self. intentos = Self. intentos + 1
if Self. intentos >= 3:
    Self.boton_entrar. Config( State = "disabled")
    Self.mensaje. Config( text = "3 intentos fallidos. Espera 30 segundos")
    Self. ventana. After (30000, self. desbloquear)
else:
    Self. mensaje. Config(tex = "usuario o contraseña incorrectos")

#Vuelve a activar el botón después de esperar
def desbloquear (Self):
    Self.intentos = 0
    Self. boton_entrar. Config( State = "normal", text = "Entrar >")
    Self. mensaje. Config ( text = " ")
#Crea una nueva con lo que está escrito en las cajas
def registrar (Self):
    Usuario = Self. caja_usuario. get(). Strip()
    Clave = Self. caja_clave. get()
    if Usuario = = " " or Clave = = " ":
        Self. mensaje. Conf.g(text = "Llena todos los campos para registrarte")
    elif " " in Usuario or "," in Usuario or "," in Clave:
        Self. mensaje. Conf.g(text = "El usuario debe tener mínimo 4 letras")
    elif Len(Clave) < 6:
        Self. mensaje. Config(tex= "La contraseña debe tener mínimo 6 -
        Caracteres")
    elif Clave . isdigit() or clave. isalpha():
        Self. mensaje. Config( text = "la contraseña debe tener letras y-
        números")
    elif Usuario in Self. Usuarios. Lista:
        SelfMensaje. configuración ( text = "Ese usuario ya existe")
    el Se:
        self. Usuarios. guardar (Usuario, Clave, Palabra. Strip())
        Self.mensaje. Config (text = " ")
        messagebox. Showinfo ("Registro", "Usuario creado, ya puedes entrar")
#Muestra la contraseña si la palabra secreta es correcta
def olvide_clave (Self):
    Usuario = SimpleDialog. askstring ("Recuperar", "Escribe tu usuario:")
    if usuario = = None:
        return
    Palabra = SimpleDialog askstring ("Recuperar", "Escribe tu palabra -
    secreta:")
    if   palabra = = None:
    Return
```

Scanned with CamScanner

```
    Usuario = usuario.strip()
if usuario in self.usuarios.lista and self.usuarios.lista[usuario][1] == palabra.
strip():
messagebox.showinfo("Recuperar", "Tu contraseña es: " + self.usuarios.lista-
[usuario][0])
else:
    messagebox.showerror("Recuperar", "Usuario o palabra secreta incorrecto")

ventana = tk.Tk()
Login(ventana)
ventana.mainloop()
```

Scanned with CamScanner